



Nom : **DJINE**

Prénoms : **Sinto Pafing**

Matricule : **23U2292**

Année académique : **2025–2026**

Architecture logicielle et conception

TP 0 ICT301 : Principes SOLID

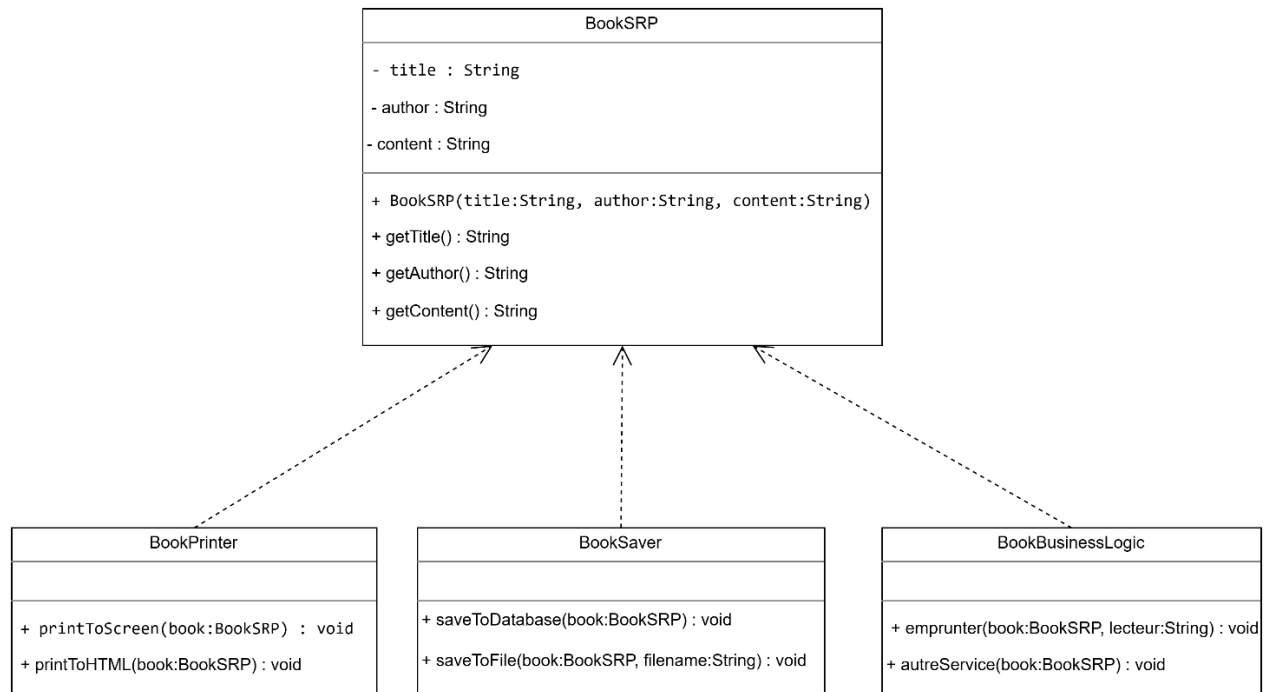
1. SRP : Single Responsibility Principle

➤ Avant refactoring

Book
- title : String - author : String - content : String
+ Book(title:String, author:String, content:String) + getTitle() : String + getAuthor() : String + getContent() : String + printToScreen() : void + saveToDatabase() : void + emprunter(lecteur:String) : void

Note : La classe Book viole le SRP car elle regroupe plusieurs responsabilités : données, présentation, persistance et logique métier.

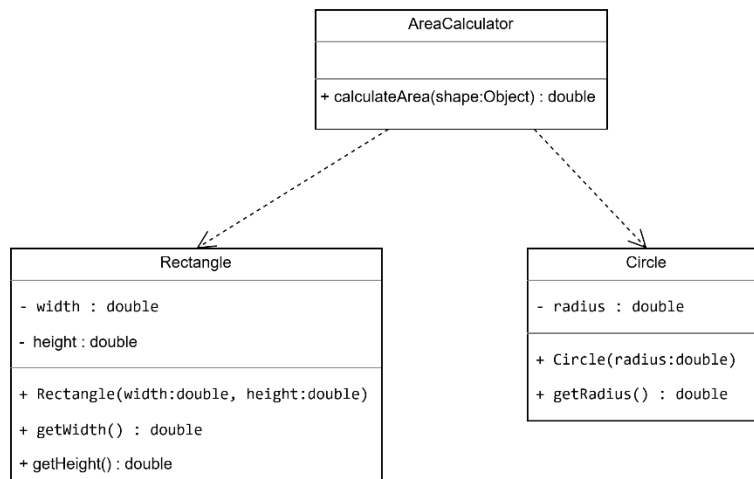
➤ Après refactoring



Note : Les responsabilités sont séparées : BookSRP gère les données, BookPrinter l’affichage, BookSaver la sauvegarde, et BookBusinessLogic la logique métier. Chaque classe a une seule raison de changer.

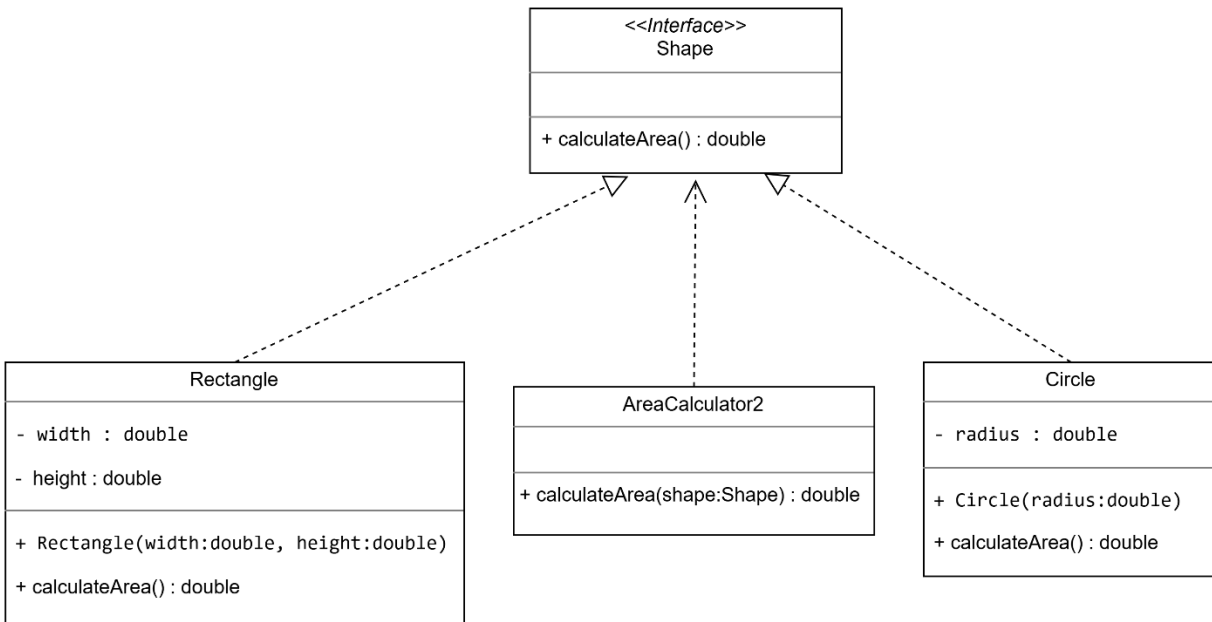
2. OCP : Open–Closed Principle

➤ Avant refactoring



Note : Le code n'est pas fermé à la modification : ajouter une nouvelle forme oblige à modifier AreaCalculator (tests instanceof) → violation du OCP.

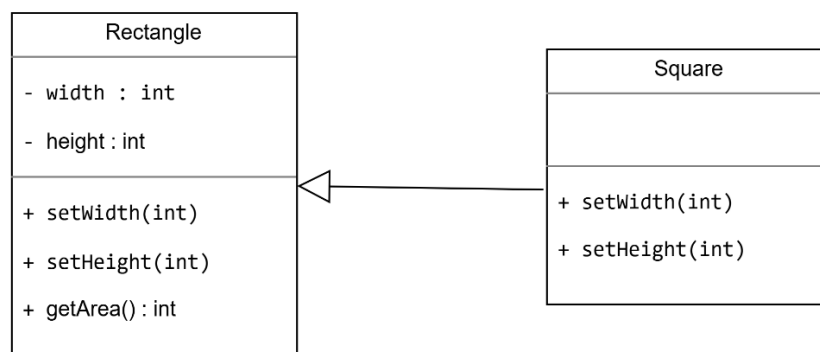
➤ Après refactoring



Note : Avec l'interface Shape, on peut ajouter une nouvelle forme (ex: Triangle) en créant une nouvelle classe qui implémente Shape, sans modifier AreaCalculator2 donc OCP respecté.

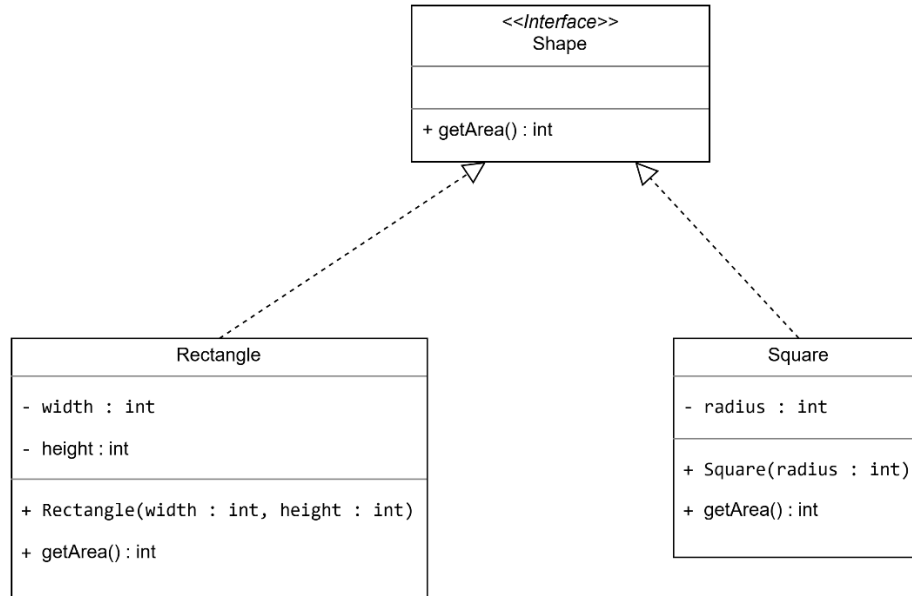
3. LSP : Liskov Substitution Principle

➤ Avant refactoring



Note : En utilisant un carré à la place d'un rectangle, le comportement attendu est modifié (aire incorrecte). Le principe de substitution est violé.

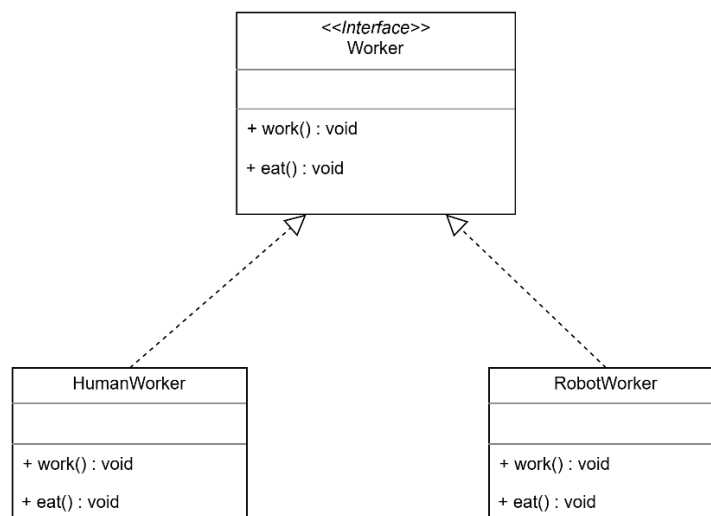
➤ Après refactoring



Note : Toutes les implémentations de Shape sont substituables sans modifier le comportement du programme.

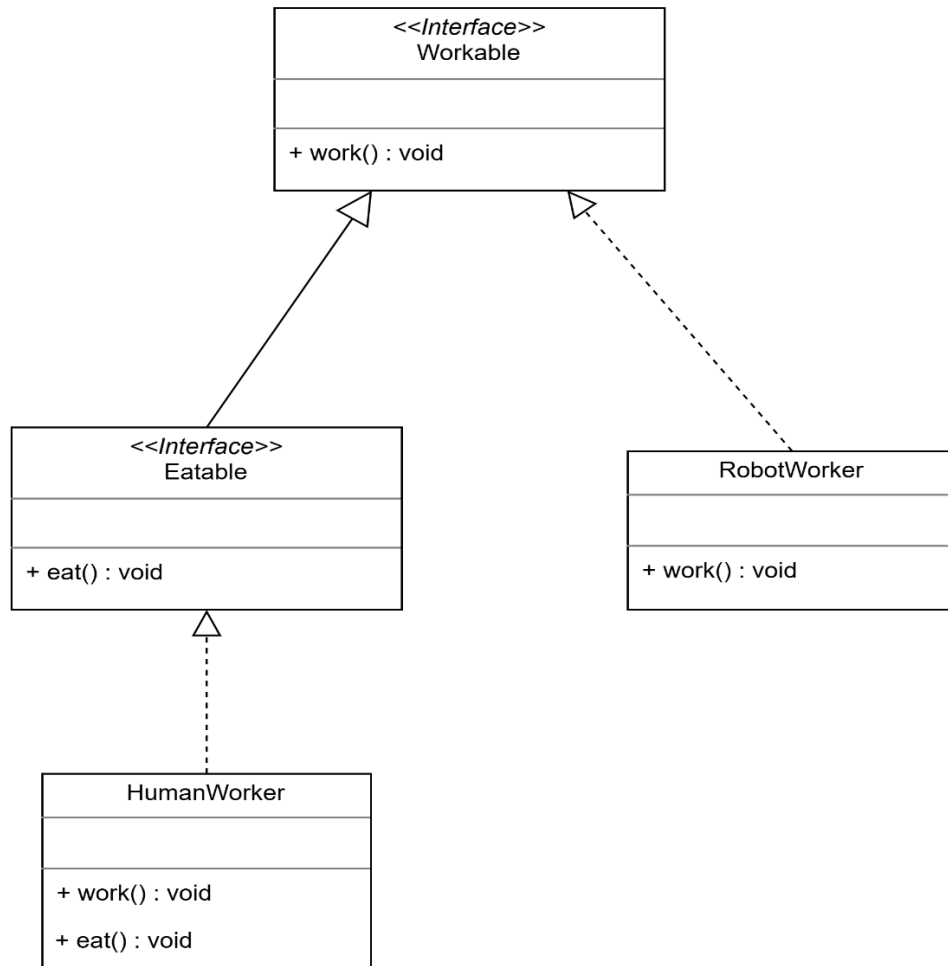
4. ISP : Interface Segregation Principle

➤ Avant refactoring



Note : L'interface Worker est trop large : le robot est obligé d'implémenter la méthode eat() qu'il n'utilise pas, ce qui viole le principe ISP.

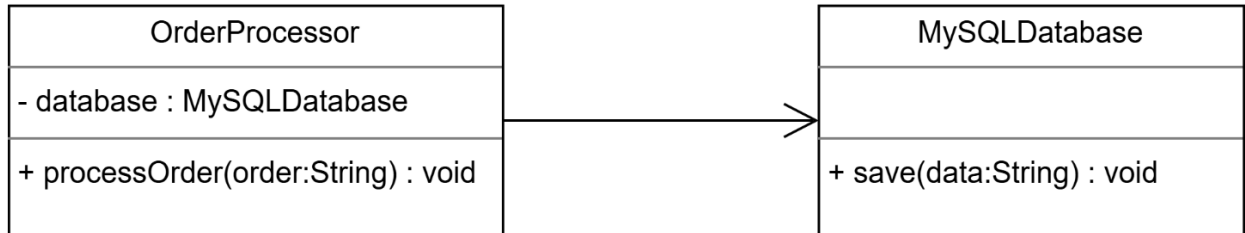
➤ Après refactoring



Note : Les interfaces sont désormais spécialisées : chaque classe dépend uniquement des méthodes dont elle a besoin, conformément au principe ISP.

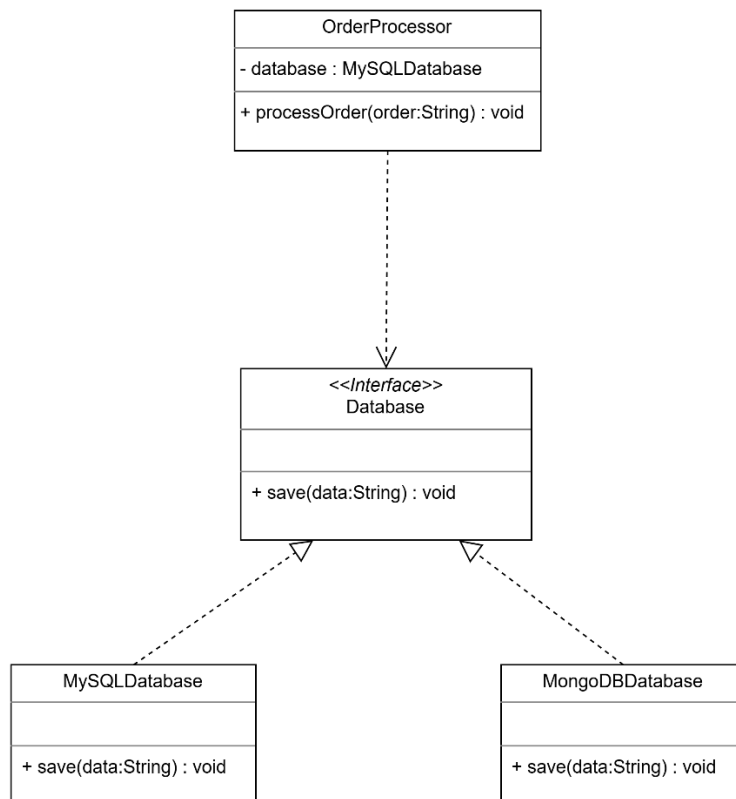
5. DIP : Dependency Inversion Principle

➤ Avant refactoring



Note : Le module métier **OrderProcessor** dépend directement d'une implémentation concrète (**MySQLDatabase**), ce qui viole le principe d'inversion des dépendances.

➤ Après refactoring



Note : Le module métier dépend désormais d'une abstraction (**Database**). Les détails d'implémentation dépendent de l'interface, ce qui respecte le principe DIP.